

Effect of VM Memory Size on Paging, Swapping and Thrashing in a VMware Ubuntu Guest

Name: MOMOTAZ JAHAN MOMO ID: 202414061
CSE-307: Operating Systems, Section B
GitHub: <https://github.com/YOUR/REPO>

4 October 2026

Abstract

We run one fixed memory-intensive workload in an Ubuntu 26.04 guest under VMware Workstation while changing only the VM's RAM (4096, 2048, 1536 and 1024 MB). We record runtime, major page faults and swap traffic. With 4096 MB the workload finishes in a few seconds; at 2048 MB it takes about 9 minutes, and at 1536 and 1024 MB most runs do not finish within a 900 s limit. The results show the transition from normal paging to thrashing when the working set no longer fits in guest memory.

1 Introduction

Modern systems rely on virtual memory: pages that do not fit in RAM are moved to a swap area and brought back on demand [1]. When the memory demand of the running processes exceeds physical memory, the system can spend most of its time moving pages instead of computing, which is called thrashing [4]. In a virtual machine, the guest's RAM is a quota set by the hypervisor, so shrinking the VM changes the memory pressure without changing the program. The goal of this paper is to observe this effect experimentally and explain it with the concepts from the course.

2 Background

Demand paging. A page is loaded only when referenced; a reference to a non-resident page causes a page fault, and a *major* fault requires disk I/O [1, 2]. **Swapping.** Under memory pressure the kernel writes (swap-out) anonymous pages to the swap area and reads them back (swap-in) when needed. Linux exposes these counters as `pswpin`, `pswpout` and `pgmajfault` in `/proc/vmstat` [5]. **Working set and thrashing.** A process runs efficiently only if its working set fits in memory [3]; otherwise page-fault rate rises sharply and CPU utilisation drops [4]. **Virtualization.** The hypervisor presents a fixed amount of RAM to the guest. The guest kernel manages it with its own paging, and guest swap I/O goes to a virtual disk backed by the host's storage [6].

3 Experimental Setup

Platform. Host: Dell Latitude 3450, 13th Gen Intel Core i5-1335U (1.30 GHz, 10 cores / 12 logical processors), 8 GB RAM (7.69 GB usable), 477 GB SSD, Windows, VMware Workstation 26.0.0.25388281. Guest: Ubuntu 26.04.1 LTS, kernel 7.0.0-38-generic, 2 vCPUs, 2 GB swapfile, `vm.swappiness=60`, Python 3.14.4, NumPy 2.3.5.

Workload. The same script in all runs: allocate and touch a 1500 MB int64 array, perform 3 sequential sweeps (one access per 4 KB page), then 2×10^7 random reads (seed 42). The array is larger than the usable RAM of the 2048 MB and smaller VMs.

Procedure. For each configuration: set RAM in VMware, power off/on, run the script three times. Before each run caches are dropped and swap is reset. `vmstat 1` runs in parallel and `/usr/bin/time -v` measures the process. Counters in `/proc/vmstat` are read before and after. A run is killed after 900 s (`timeout`); such runs are marked as not finished and their counters cover only the first 900 s.

Table 1: VM RAM (VMware setting) and usable guest memory reported by `free -m`.

VM RAM (MB)	Usable (MB)	Mode
4096	≈ 3350	GUI (original VM)
2048	≈ 1594	text, clone
1536	n/a	text, clone
1024	≈ 907	text, clone

4 Results

Table 2: Mean over 3 runs. $\geq$ marks configurations where runs hit the 900 s limit.

VM RAM	Finished	Elapsed (s)	Max RSS (MB)	pswpin	pswpout	Major faults
4096	3/3	2.4	1549	915	125,235	937
4096 (repeat)	3/3	7.9	1547	31,871	115,776	5,028
2048	3/3	557.9	1237	7,504,620	4,255,176	4,192,310
1536	1/3	≥ 848.3	1233	6,348,359	3,514,077	3,474,728
1024	0/3	≥ 900.2	568	6,898,506	6,607,944	5,156,843

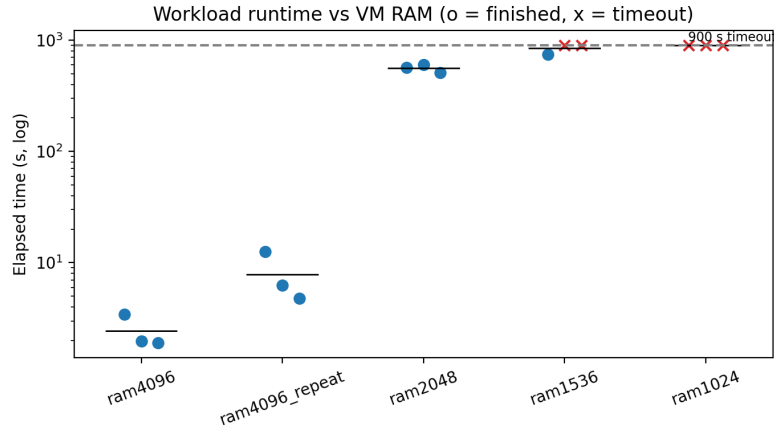


Figure 1: Elapsed time per run (log scale). Crosses are runs killed at 900 s.

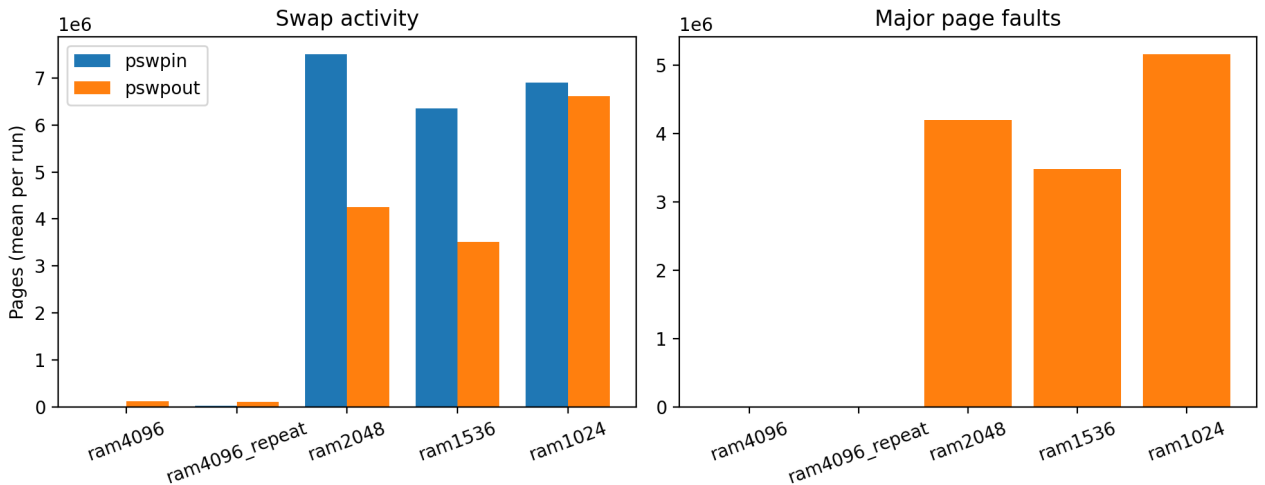


Figure 2: Mean swap-in/out pages and major page faults per configuration.

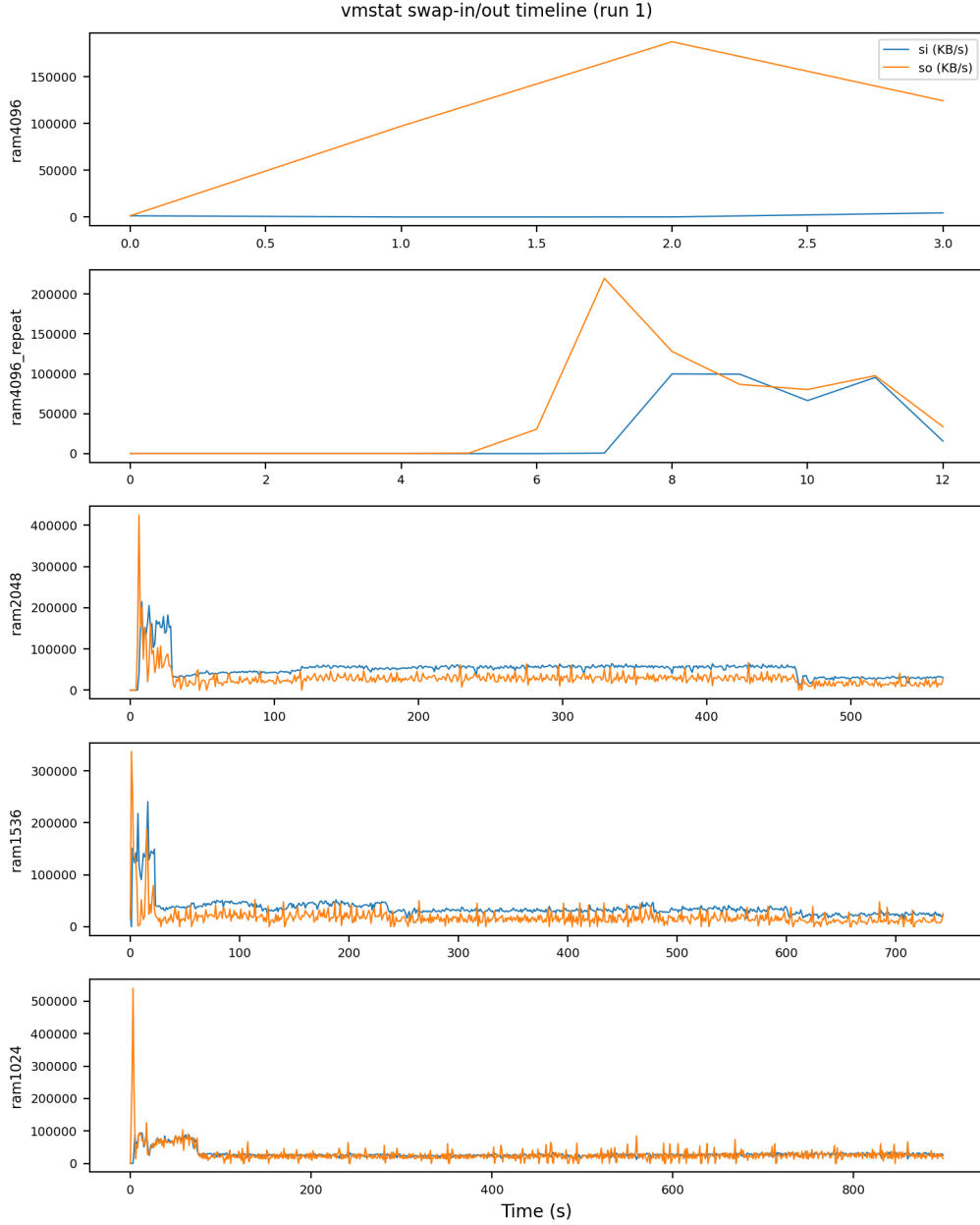


Figure 3: `vmstat` swap-in (si) and swap-out (so) over time, run 1 of each configuration.

5 Analysis

4096 MB. The 1500 MB array fits in the usable memory, so the job completes in 2–8 s with only hundreds to a few thousand major faults. Some swap-out still occurs (~ 0.1 M pages, about 450–500 MB) because the desktop and cached pages share the 3350 MB. The repeat set was slower (7.9 s) with more swap-ins, showing run-to-run variability from the guest’s page-cache and swap state.

2048 MB. Usable memory is about 1594 MB, barely above the array size plus the OS, so pages are evicted while still needed. Runtime rises by about $230\times$ (to ≈ 558 s) and major faults to 4.2M. The `vmstat` trace shows free memory dropping to about 64 MB and then swap-out of 114–425 MB/s with 15–33% I/O wait. This is the onset of thrashing.

1536 and 1024 MB. Runs mostly exceed 900 s. At 1024 MB the process resident size falls to about 570 MB, and in run 1 the job spent 322 s of system time against 8.6 s of user time (36% CPU): the kernel’s fault and swap handling dominated. Over 6.5M pages were swapped in and out within 900 s, i.e. about 25 GB of swap traffic each direction for a 1.5 GB data set.

Why. The access pattern sweeps the whole array and then reads it at random, so the working set is the whole array. Once memory is smaller than the working set, almost every random access misses, and each miss

costs a disk access that is orders of magnitude slower than RAM [1, 4]. Runtime therefore grows non-linearly as RAM shrinks rather than in proportion to the lost memory. Because the guest swap file lives on a virtual disk, the hypervisor and host storage add further latency.

6 Limitations

(1) The 4096 MB runs were executed on the original VM with the desktop running, while the other sizes were run on a linked clone in text mode (to avoid GUI soft lockups), so the 4096 MB comparison has a mode confound; this would, if anything, slow the 4096 MB case. (2) Only three runs per configuration and a 900 s cap, so timed-out configurations give lower bounds; counters of killed runs cover only 900 s. (3) vCPU count was not varied. (4) One host machine; other host load was not controlled. (5) Usable memory differs from the VMware setting because of kernel reservations. (6) The host has only 8 GB of RAM, so a 4096 MB guest leaves roughly half of host memory to Windows; host memory pressure (and host paging) may have affected guest I/O, especially in the swap-heavy runs.

7 Conclusion

Reducing guest RAM below the working set of a workload converts a seconds-long job into thrashing: from 2.4 s at 4096 MB to about 558 s at 2048 MB and no completion within 900 s at 1024 MB. Memory sizing of VMs must therefore consider the working set, not the average usage.

References

- [1] A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, Wiley (use your edition; Ch. Virtual Memory).
- [2] A. S. Tanenbaum, H. Bos, *Modern Operating Systems*, Pearson (use your edition).
- [3] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, 11(5), 323–333, 1968.
- [4] P. J. Denning, “Thrashing: Its causes and prevention,” *Proc. AFIPS Fall Joint Computer Conference*, 1968.
- [5] The Linux Kernel documentation, “Documentation for /proc/sys/vm/” and /proc/vmstat, <https://docs.kernel.org>.
- [6] VMware Workstation Pro documentation, memory configuration of virtual machines, <https://docs.vmware.com>.